

# CS & IT ENGINEERING



## Operating System

Process Synchronization

Lecture - 4

By- Vishvadeep Gothi sir



# Recap of Previous Lecture



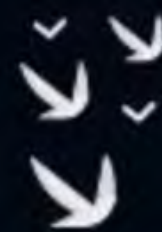
doubt?

Topic

2-Process Solutions



# Topics to be Covered



Topic

Hardware Solutions





## Topic : Solution of Critical Section Problem

### **Mutual Exclusion:**

If one process is executing the critical section, then other process is not allowed to enter into critical section.



## Topic : Solution of Critical Section Problem



### Progress:

If no any process is in critical section and any process wants to enter into critical section, then the process must be allowed.



## Topic : Solution of Critical Section Problem

### **Bounded Waiting:**

If a process  $p_1$  is executing in critical section and other process  $p_2$  is waiting for critical section, then the waiting time of  $p_2$  must be bounded. Which means  $p_1$  must not enter in to critical section again and again by keeping  $p_2$  in waiting for long.

## Solution 1

✓ starvation

Boolean lock=~~false~~; ~~True~~  
~~P0~~ ~~F~~ ~~T~~

```
while(true)
{
    while(lock);
    lock=true;
    //CS
    lock=false;
    RS;
}
```

P1

```
while(true)
{
    while(lock);
    lock=true;
    //CS
    lock=false;
    RS;
}
```

## Solution 2

✓ Starvation

```
int turn=0;
```

```
while(true)
{
    while(turn!=0);
        CS
    turn=1;
    RS;
}
```

P<sub>1</sub>

```
while(true)
{
    while(turn!=1);
        CS
    turn=0;
    RS;
}
```

## Peterson's Solution

$Flag[0] = \cancel{F} \cancel{T} \cancel{F} T$   
 $Flag[1] = \cancel{F} T$

Boolean Flag[2]={False, False};  
 int turn;

*P0*

while(true) {  
     Flag[0]=true;  
     turn=1;

→ while(Flag[1] && turn==1);

→ CS

    Flag[0]=False;  
     RS;

}

Turn = ~~1~~  
           1

*P1*

while(true){  
     Flag[1]=true;  
     turn=0;

→ while(Flag[0] && turn==0);

→ CS

    Flag[1]=False;  
     RS;

}

✗ Starvation

H.W.

## Question 1

turn=0;

```
while(true)
{
    while(turn);
    turn=1;
    //CS
    turn=0;
    RS;
}
```

```
while(true)
{
    while(turn);
    turn=1;
    //CS
    turn=0;
    RS;
}
```

## Question 2

H. W.

```
lock=False;
```

```
while(true)
{
    while(lock!=False);
    CS
    lock=True;
    RS;
}
```

```
while(true)
{
    while(lock!=True);
    CS
    lock=False;
    RS;
}
```



## Topic : Synchronization Hardware

↓  
Some inst<sup>ns</sup> are implemented in Computer architecture to provide synchronization.

1. TestAndSet()
2. Swap()



## Topic : TestAndSet()



Returns the current value flag and sets it to true.

TestAndSet(Lock) → if lock is false  
→ It returns false  
but makes lock True }

---

→ If lock is True  
→ It returns True  
and makes lock True



## Topic : TestAndSet()



*lock = ~~false~~ ~~true~~  
true false!*

Boolean Lock=False;

```
boolean TestAndSet(Boolean *trg){  
    boolean rv = *trg;  
    *trg = True;  
    Return rv;  
}
```

```
while(true)    P0    P1  
{  
    while(TestAndSet(&Lock));  
        CS  
    Lock=False;  
}
```

*✓ starvation  
✓ mutual Exclusion  
✓ progress  
✗ Bounded waiting*



## Topic : Swap()



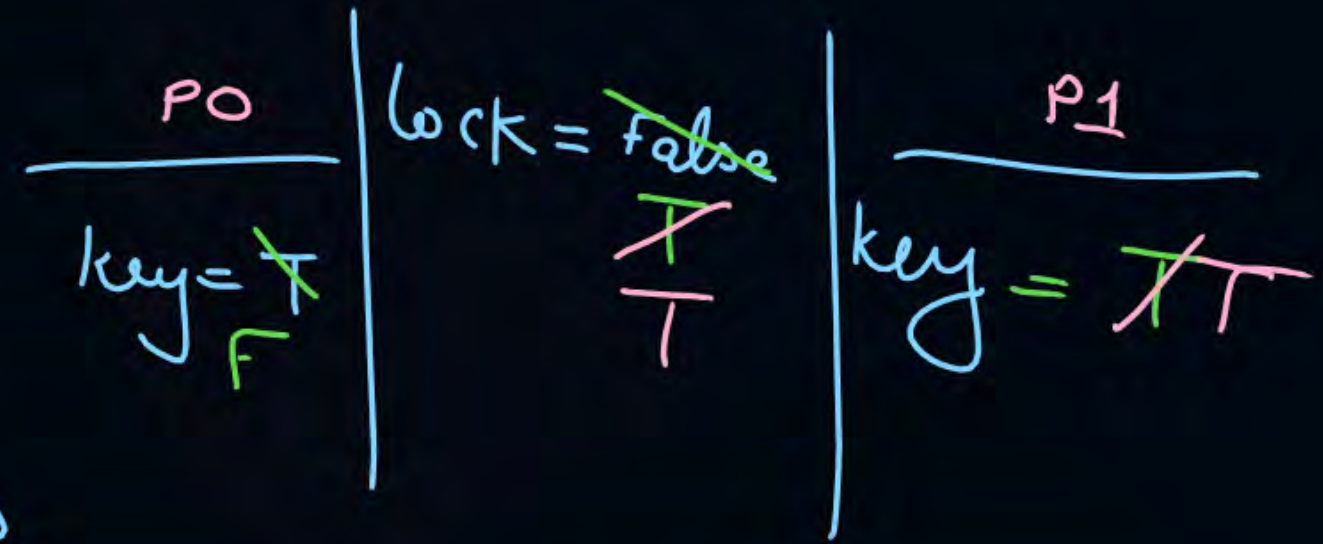
Boolean Key;

Boolean Lock=False;

void Swap(Boolean \*a, Boolean \*b)

```
{  
    boolean temp = *a;  
    *a=*b;  
    *b=temp;  
}
```

//Local to each process  
//Shared



```
while(true){
```

```
    Key = True;
```

```
    while (key==True)
```

```
        Swap(&Lock, &Key);
```

CS

```
    Lock=False;
```

```
}
```

✓ Starvation

✓ M.E.

✓ Progress

X B.W.

starvation in synchronization :-

starvation

In CPU scheduling algo  $\Rightarrow$  Process does not get CPU for indefinite time.

In synch.  $\Rightarrow$  Process does not get C.S. for indefinite time.



## Topic : Synchronization Tool

1. Semaphore ✓
2. Monitor ✗

↓  
Tools used by programmers to add  
synch. in their code.



## Topic : Synchronization Tool

Semaphore

→ (non-negative)

- Integer value which can be accessed using following functions only

1. wait() / P() / Degraded()
  2. signal() / V() / Upgrade()
- } both are atomic functions



## Topic : wait() & signal()

Semaphore S



```
wait(S)
```

```
{  
    while(S<=0);  
    S--;  
}
```

```
signal(S)
```

```
{  
    S++;  
}
```





## Topic : Types of Semaphore

| Binary Semaphore                          | Counting Semaphore                          |
|-------------------------------------------|---------------------------------------------|
| <p>↓</p> <p>accepts values<br/>0 or 1</p> | <p>↓</p> <p>0, 1, 2, 3, 4, 5, . . . . .</p> |



## Topic : Types of Semaphore

| Binary Semaphore                                                                          | Counting Semaphore                                                     |
|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| It is used to implement the solution of critical section problems with multiple processes | It is used to control access to a resource that has multiple instances |



## Topic : wait() & signal() on Binary Semaphores

Binary semaphore

$S = 1$

$\text{wait}(S) \Rightarrow$  successfully completed and  $S = 0$

$S = 0$

$\text{signal}(S) \Rightarrow$  successfully completed and  $S = 1$

$S = 0$

$\text{wait}(S) \Rightarrow$  unsuccessful

$S = 1$

$\text{signal}(S) \Rightarrow$  successfully completed and  $S = 1$



## Topic : Critical Section Solution

Binary semaphore

$S = 1$

$P_0$   $P_1$

while(True)

{

→ wait(S)

C.S.

signal(s)

}

Mutual Exclusion

$S = 1$

CS is open

$S = 0$

CS is closed

$S = \cancel{1} \cancel{0} 1$



**2 mins Summary**

**Topic**

**Hardware Solutions**





**Happy Learning**

**THANK - YOU**

